

William Jackson

Senior Software Engineer – React C# (.NET) Azure/AWS

Wellington, Nevada, United States | +1 (786) 949-7561 | williamjdev@outlook.com | linkedin.com/in/william-jackson-40036a3ba

PROFESSIONAL SUMMARY

Senior Software Engineer with ~10 years of experience building **SaaS, fintech-oriented**, and operational workflow platforms with **React 16–18, TypeScript 4–5**, and **C# / .NET / ASP.NET Core 3.1–8** across cloud-native environments. Strong track record delivering **API-first architectures, BFF gateways**, and resilient event-driven services that stay stable under retries, failures, and high concurrency. Experienced modernizing legacy **.NET Framework 4.6–4.8** systems into modern LTS platforms, improving reliability, observability, and release confidence through structured telemetry, idempotency, and performance tuning. Known for solving production defects, hardening security-sensitive workflows, and shipping maintainable systems for high-availability business applications.

SKILLS

- **Frontend** — **React 16–18, Next.js 12–14, TypeScript 4–5, Redux Toolkit, React Query, React Native, HTML5, CSS3, Tailwind CSS, Bootstrap, Webpack, Vite, Storybook, WCAG / ARIA, SSR / ISR** patterns, micro-frontend integration, bundle optimization, code-splitting, memoization, performance profiling
- **Backend & APIs** — **C#, .NET Framework 4.6–4.8, .NET / ASP.NET Core 3.1–8, ASP.NET MVC 5, ASP.NET Web API 2, REST APIs, GraphQL, minimal APIs, microservices, background workers, Hosted Services, WebSockets**, API versioning, deprecation strategy, rate limiting, idempotency, deduplication, secure middleware, validation pipelines, distributed systems
- **Data, Messaging & Search** — **PostgreSQL, MySQL, SQL Server, Redis, MongoDB, DynamoDB, AWS SQS/SNS, Kafka, RabbitMQ, Elasticsearch, OpenSearch, Redshift, BigQuery**, indexing strategy, execution-plan tuning, transaction handling, cache key/TTL design, search indexing, schema validation
- **Cloud & DevOps** — **AWS (primary): EKS, ECS, EC2, Lambda, API Gateway, S3, CloudFront, RDS, DynamoDB, ElastiCache, EventBridge, Step Functions, IAM** • **Azure: AKS, App Service, Azure SQL, Service Bus, Key Vault, Application Insights** • **GCP: GKE, Cloud Run, Pub/Sub, Cloud Storage, BigQuery** • **Docker, Kubernetes, Helm, Argo CD, GitOps, Terraform, CloudFormation, CI/CD: GitHub Actions, GitLab CI, Jenkins, Azure DevOps, CircleCI** • release strategies (blue/green, canary), secrets management
- **Observability, Quality & Security** — **OpenTelemetry, Datadog, Prometheus, Grafana, ELK/EFK, CloudWatch**, structured logging, correlation IDs, SLO/SLI alerting, Testing: **xUnit/NUnit, Jest, React Testing Library, Cypress, Playwright**, API contract tests (Pact-style), Security: **OAuth2, OpenID Connect, JWT, Okta, Auth0, AWS Cognito**, rate limiting, **RBAC/ABAC**, audit logging, secure storage, threat modeling basics

WORK HISTORY

Senior Software Engineer

February 2021 to December 2025

Viaro Networks Inc. - Delaware, United States

- Architected a large-scale **React 17–18** and **TypeScript 4–5** frontend refresh for a SaaS operations platform, separating container logic from presentational components and reducing UI regression defects by **38%** through stricter state isolation and reusable rendering contracts.
- Designed a **.NET / ASP.NET Core 6–8** BFF gateway that normalized payload shapes, pagination rules, and error semantics, which protected frontend screens from upstream contract drift and made release coordination substantially easier across distributed teams.
- Resolved a high-impact production defect where locale-sensitive number parsing corrupted settlement totals in edge markets, fixing the issue with invariant serialization rules, canonical decimal handling, and targeted regression tests around money and timezone conversions.
- Eliminated duplicate-write incidents caused by rapid user retries and event replays by implementing **idempotency keys**, deduplication checks, and replay-safe consumer logic across API and queue-processing layers, reducing reconciliation noise by **45%**.
- Built durable asynchronous workflows using **AWS SQS/SNS** and background workers for long-running financial and reporting tasks, ensuring jobs could recover safely after restarts without partial side effects or inconsistent downstream state.

- Introduced **Kafka** fan-out pipelines for operational events and audit propagation, applying deterministic partition keys, retry policies, and DLQ isolation so poison messages were quarantined instead of stalling the broader processing stream.
- Led migration of legacy **.NET Framework 4.7/4.8** service modules into modern **.NET 6–8** services, replacing fragile shared-state patterns and scheduler glue with hosted workers, smaller deployable units, and cleaner dependency boundaries.
- Improved p95 API latency by **34%** through **PostgreSQL** query-plan tuning, composite index refinement, and carefully scoped **Redis** caching, while using explicit invalidation and TTL strategies to protect correctness on read-heavy endpoints.
- Strengthened platform security by implementing **OAuth2 / OIDC / JWT** flows with consistent authorization middleware, aligning **RBAC / ABAC** checks to real operational roles and ensuring audit events reflected both UI actions and backend decisions.
- Prevented burst-related throttling failures by adding gateway-level rate limiting, adaptive retry policies, and request shaping controls, which stabilized peak traffic handling and improved error-rate behavior during partner spikes by **30%**.
- Rolled out end-to-end **OpenTelemetry** instrumentation from browser interactions through **ASP.NET Core**, queue consumers, and storage boundaries, making incident triage materially faster through trace correlation, metric baselines, and linked logs.
- Standardized delivery using **Docker, Kubernetes, Helm**, and infrastructure-as-code on **AWS** and **Azure**, giving teams safer promotion paths, least-privilege deployment patterns, and repeatable environment setup for complex multi-service releases.

Software Engineer
Avantia, Inc. - Ohio, United States

October 2018 to January 2021

- Built **React 16–17** operational consoles for internal and partner-facing workflows, giving support teams live status visibility and guided remediation paths that reduced manual intervention across common failure scenarios by **32%**.
- Implemented **ASP.NET Core 3.1–5** REST APIs with stable pagination, explicit validation rules, and predictable error taxonomies, allowing frontend teams to build resilient screens without hidden assumptions about backend behavior.
- Fixed a costly retry bug where partner-side resubmissions produced duplicate tickets and inconsistent statuses, solving it through idempotency tokens, audit trails, and conflict-aware handlers that preserved exactly-once business intent.
- Migrated blocking synchronous processing into queue-backed background jobs with progress tracking and user-facing status feedback, which removed timeout-heavy interactions from the UI and improved completion reliability by **40%**.
- Improved data integrity across **PostgreSQL, MySQL, and SQL Server** by introducing tighter migration discipline, rollback planning, index cleanup, and verification scripts that caught schema drift before it affected production workloads.
- Added adapter layers around third-party integrations so provider-specific quirks, schema oddities, and temporary outages were isolated within bounded modules rather than leaking instability into the application core.
- Hardened authentication and authorization flows using **JWT**, role-based access checks, and policy-driven endpoint protection, which reduced permission mismatches and made security behavior more consistent across administrative surfaces.
- Implemented centralized exception handling and stable error-code mapping so **React** screens could display actionable messages instead of generic failure banners, improving support diagnostics and shortening troubleshooting time for end users.
- Expanded automated test coverage around brittle business rules, integration edges, and failure-prone retry paths, which reduced escaped regressions and increased deployment confidence for high-change operational features.
- Introduced structured logging, correlation identifiers, and early trace baselines through **Application Insights** and service-level instrumentation, turning fragmented incident analysis into a more searchable and connected debugging workflow.

- Optimized hot endpoints by removing N+1 access patterns, reducing chatty repository calls, and tightening serialization overhead, which improved response times by **27%** and made peak operational load more predictable.

Software Developer
ArnAmy, Inc. - Florida, United States

March 2016 to September 2018

- Delivered internal dashboards with **React 15–16** that replaced spreadsheet-driven workflows, giving operations teams clearer visibility into reconciliation, reporting, and exception queues while improving auditability for day-to-day actions.
- Built **C#, ASP.NET Web API 2**, and early **ASP.NET Core** services that unified multiple upstream sources behind cleaner contracts, reducing duplicated business logic and making downstream consumers less sensitive to source variation.
- Fixed a severe reconciliation issue caused by partial updates and interrupted job chains, resolving it with stronger transaction boundaries, compensating actions, and state verification checks that reduced data correction work by **35%**.
- Implemented request validation and predictable error envelopes so clients could distinguish user mistakes, transient failures, and data issues more clearly, which lowered “silent failure” support cases and improved operational trust.
- Automated scheduled processing and reporting jobs with retries, exception summaries, and alerting hooks, replacing manual execution patterns with deterministic nightly workflows that stakeholders could depend on during business-critical cycles.
- Improved reporting performance through slow-query profiling, targeted indexing, and execution-plan review in **SQL Server** and **PostgreSQL**, cutting timeout frequency significantly during high-volume reporting windows and monthly close activities.
- Added structured logging and correlation identifiers across API calls, background processes, and persistence operations, making root-cause analysis faster when failures crossed more than one service or data boundary.
- Introduced schema validation and quarantine handling for malformed inbound records, preventing poison data from propagating into downstream tables and giving operations teams safer recovery workflows for bad-source scenarios.
- Integrated **MongoDB** for flexible document-oriented use cases while keeping relational stores authoritative for transactional paths, enabling the team to support evolving payload structures without sacrificing reporting consistency.
- Implemented **Redis** caching and **Elasticsearch** indexing for read-heavy screens and fast filtering, improving dashboard responsiveness by **28%** while keeping cache invalidation and search synchronization rules explicit and maintainable.

Junior Software Developer
Centric Tech Inc. - New Jersey, United States

October 2014 to February 2016

- Built early internal UI modules with **React 0.14–15** style patterns and API-driven screens, improving workflow throughput incrementally without forcing a risky full-platform rewrite across already active operational teams.
- Implemented **ASP.NET MVC 5** and **Web API 2** controllers with consistent validation and error handling, giving client applications clearer contracts and reducing ambiguity that previously created avoidable support and QA churn.
- Fixed a critical SQL Server consistency defect by tightening transaction handling and save-order rules, preventing partial writes that had required manual cleanup and reducing repeat correction incidents by **25%**.
- Standardized CI execution in **Jenkins** by codifying build steps, dependency restoration, environment variables, and packaging behavior, which eliminated several machine-specific failures that had slowed development and release flow.
- Added unit tests around fragile business rules and data-mapping logic, improving regression detection earlier in the cycle and reducing the team’s reliance on production hotfixes for routine release stabilization.

- Diagnosed slow endpoints using query inspection, application logs, and payload analysis, learning practical tuning techniques that improved API responsiveness and informed later optimization work across more complex systems.
- Supported queue-backed processing with **RabbitMQ** semantics and safer retry/backoff handling, helping the platform recover more gracefully from transient failures and reducing stuck-job incidents during bursty internal workloads.
- Implemented basic **Redis** caching for repeated lookups and high-read views, lowering unnecessary database pressure while using explicit TTL rules to avoid stale results on frequently changing operational data.
- Delivered early **Elasticsearch** indexing for internal search and filtering scenarios, giving users faster access to operational records without forcing expensive scans against transactional database tables.

EDUCATION

Bachelor's degree in Computer Science

2010 to 2014

Stanford University Department of Computer Science

- Built strong foundations in **algorithms**, **databases**, and **software engineering**, applying coursework through practical team projects and production-style system design.

CERTIFICATIONS

- **Microsoft Certified: Azure Developer Associate**
- **AWS Certified Developer – Associate**
- **Certified Kubernetes Application Developer (CKAD)**
- **HashiCorp Certified: Terraform Associate**